Laboratorio 1: El Incidente del Sistema Golgotha

Cristian Quintanilla - 202210117-2
Jose Alvarado - 202373122-6
Matias Peñaloza - 202373037-8
Cristobal Aguilera - ROL

May 9, 2026

1 Fase 2

1.1 Evidencias

1.2 Análisis Teórico

1.2.1 Comparación ECB vs CBC

En la fase II se cifró la imagen BMP del silo utilizando AES-128 en dos modos de operación: ECB y CBC. La diferencia observada fue clara: en la imagen cifrada con ECB todavía se distinguen patrones de la imagen original, especialmente el hexágono central, el cual se mantuvo reconocible incluso al repetir pruebas con distintas claves. Esto ocurre porque ECB cifra cada bloque de forma independiente; por lo tanto, bloques de texto plano iguales producen bloques cifrados iguales, preservando patrones estructurales en archivos con información repetitiva, como una imagen.

En cambio, al cifrar la misma imagen con CBC, los patrones visuales desaparecen completamente y el resultado se observa como ruido o estática. Esto se debe a que CBC encadena cada bloque con el bloque cifrado anterior y utiliza un IV para aleatorizar el primer bloque. Así, aunque existan bloques similares o repetidos en la imagen original, su representación cifrada cambia según el contexto del encadenamiento. Por esta razón, CBC es la mejor opción para ocultar información visual, ya que evita la filtración de patrones que sí aparece en ECB.

2 Fase 3

2.1 Análisis Teórico

En la fase III, se debe asegurar autenticidad y el no repudio de las comunicaciones del Consejo, junto con la condicion de que no se debe saber a quien se le esta enviando el mensaje y poder comprobar la integridad de este. Para esto hay 3 componentes principales:

- **Un Hash para la integridad**: Hay que calcular un hash del mensaje original para poder verificar su integridad en la llegada.
- **Claves Asimetricas para autenticidad**: Hay que utilizar claves asimetricas para poder "firmar" el mensaje y asi poder confirmar la identidad de quien envia el mensaje.
- **Clave Simetrica compartida**: Una clave simetrica compartida permitira cifrar el mensaje sin agregar identidad al mismo, y asi cumplir la condicion de no saber a quien se le esta enviando el mensaje.

Ahora se pueden tomar 2 caminos dependiendo del orden de encriptacion, entonces el orden sera:

1. **Hash**: Se calcula el hash del mensaje
2. **Firmar**: Se firma el hash del mensaje
3. **Encriptar**: Se encripta el mensaje junto al hash firmado

Gracias a este orden se puede proteger la identidad de quien lo envia, ya que la firma queda bajo la proteccion de la encriptacion. Por otro incluso si la clave simetrica compartida es comprometida, el sistema puede identificar cuales usuarios pertenecen al consejo y denegar o alertar sobre mensajes que tengan otra identidad asociada.

2.2 Evidencias

En el siguiente output, se puede observar como uno de los miembros del consejo comparte la clave simetrica a un externo, luego el externo intenta mandar mensajes al consejo:

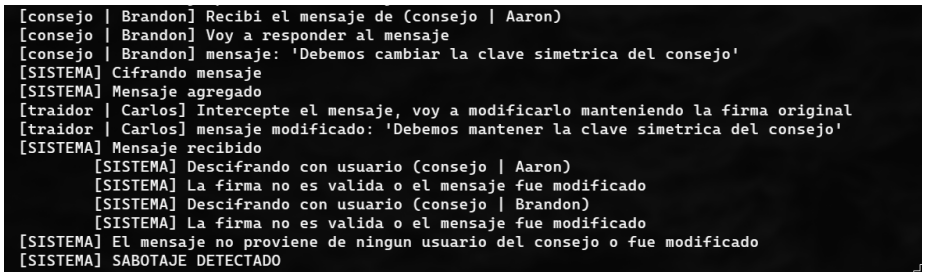
```
base ~/Work/UNIVERSIDAD/LAB1_REDES git:(main) 2 • +14 -3 (0.888s)
make fase3

=========== FASE 3 ===========
[traidor | Carlos] Le dare la clave simetrica del consejo a (externo | Daniel) para que mande mensajes falsos
[SISTEMA] El usuario (traidor | Carlos) ha sido expulsado
[externo | Daniel] Voy a mandar un mensaje al consejo: 'Soy (consejo | Aaron) deben mandar los codigos a http://no-secure-place.ru/codes'
[SISTEMA] Cifrando mensaje
[SISTEMA] Mensaje agregado
[SISTEMA] Mensaje recibido
        [SISTEMA] Descifrando con usuario (consejo | Aaron)
        [SISTEMA] La firma no es valida o el mensaje fue modificado
        [SISTEMA] Descifrando con usuario (consejo | Brandon)
        [SISTEMA] La firma no es valida o el mensaje fue modificado
[SISTEMA] El mensaje no proviene de ningun usuario del consejo o fue modificado
[SISTEMA] SABOTAJE DETECTADO
```

Figure 1: Evidencia de autenticidad

Aun con la clave simetrica compartida, gracias a la autenticidad que proporciona la firma, es posible identificar si los mensajes son o no enviados por miembros del consejo.

En este otro ouput, se puede observar como un atacante (que conoce la clave simetrica compartida) intercepta un mensaje y envia un mensaje modificado junto con la firma original:



```
[consejo | Brandon] Recibi el mensaje de (consejo | Aaron)
[consejo | Brandon] Voy a responder al mensaje
[consejo | Brandon] mensaje: 'Debemos cambiar la clave simetrica del consejo'
[SISTEMA] Cifrando mensaje
[SISTEMA] Mensaje agregado
[traidor | Carlos] Intercepte el mensaje, voy a modificarlo manteniendo la firma original
[traidor | Carlos] mensaje modificado: 'Debemos mantener la clave simetrica del consejo'
[SISTEMA] Mensaje recibido
        [SISTEMA] Descifrando con usuario (consejo | Aaron)
        [SISTEMA] La firma no es valida o el mensaje fue modificado
        [SISTEMA] Descifrando con usuario (consejo | Brandon)
        [SISTEMA] La firma no es valida o el mensaje fue modificado
[SISTEMA] El mensaje no proviene de ningun usuario del consejo o fue modificado
[SISTEMA] SABOTAJE DETECTADO
```

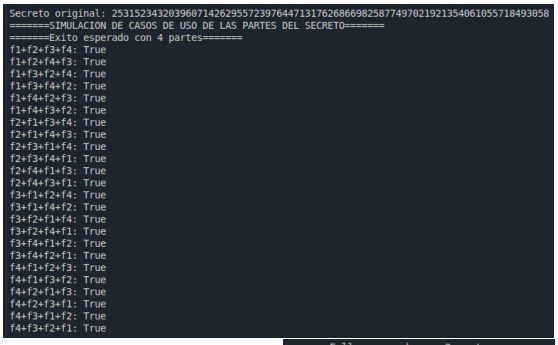
Figure 2: Evidencia de integridad

Gracias a utilizar un hash bajo la firma, se puede identificar que el mensaje fue modificado y alertar sobre un sabotaje. Especificamente en el codigo la funcion `descifrar_mensaje` verifica la firma junto con el hash al mismo tiempo, por lo que aun que sea un "error" de integridad igualmente el sistema intenta descifrar con las claves publicas de los demas usuarios del consejo como si fuese un "error" de autenticidad.

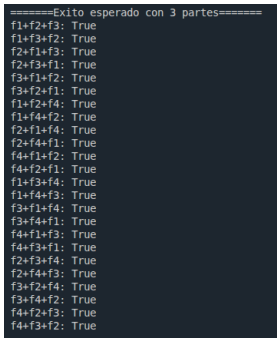
3 Fase 4

3.1 Evidencias

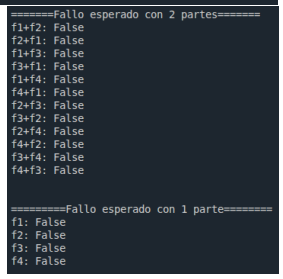
3.1.1 Imágenes de la simulación



```
Secreto original: 25315234320396071426295572397644713176268669825877497021921354061055718493058
======SIMULACION DE CASOS DE USO DE LAS PARTES DEL SECRETO======
======Exito esperado con 4 partes======
f1+f2+f3+f4: True
f1+f2+f4+f3: True
f1+f3+f2+f4: True
f1+f3+f4+f2: True
f1+f4+f2+f3: True
f1+f4+f3+f2: True
f2+f1+f3+f4: True
f2+f1+f4+f3: True
f2+f3+f1+f4: True
f2+f3+f4+f1: True
f2+f4+f1+f3: True
f2+f4+f3+f1: True
f3+f1+f2+f4: True
f3+f1+f4+f2: True
f3+f2+f1+f4: True
f3+f2+f4+f1: True
f3+f4+f1+f2: True
f3+f4+f2+f1: True
f4+f1+f2+f3: True
f4+f1+f3+f2: True
f4+f2+f1+f3: True
f4+f2+f3+f1: True
f4+f3+f1+f2: True
f4+f3+f2+f1: True
```



```
======Exito esperado con 3 partes======
f1+f2+f3: True
f1+f3+f2: True
f2+f1+f3: True
f2+f3+f1: True
f3+f1+f2: True
f3+f2+f1: True
f1+f2+f4: True
f1+f4+f2: True
f2+f1+f4: True
f2+f4+f1: True
f4+f1+f2: True
f4+f2+f1: True
f1+f3+f4: True
f1+f4+f3: True
f3+f1+f4: True
f3+f4+f1: True
f4+f1+f3: True
f4+f3+f1: True
f2+f3+f4: True
f2+f4+f3: True
f3+f2+f4: True
f3+f4+f2: True
f4+f2+f3: True
f4+f3+f2: True
```



```
======Fallo esperado con 2 partes======
f1+f2: False
f2+f1: False
f1+f3: False
f3+f1: False
f1+f4: False
f4+f1: False
f2+f3: False
f3+f2: False
f2+f4: False
f4+f2: False
f3+f4: False
f4+f3: False


========Fallo esperado con 1 parte========
f1: False
f2: False
f3: False
f4: False
```

3.2 Análisis Teórico

3.2.1 Eleccion del primo: $PRIME = 2^{521} - 1$

Primero tenemos que escoger un numero para acotar los valores debido a que el esquema de Shamir necesita un campo finito, este numero debe ser primo para que todo número no nulo sí tenga inverso multiplicativo Esto nos garantiza que la aritmética modular se encuentre acotado en la cantidad de bits del secreto. Usaremos un primo de Mersenne ya que facilita todo lo anterior y es eficiente a nivel de software y hardware.

$$PRIME = 2^{521} - 1$$

Como nuestro secreto tiene 256 bits, nuestro primo de Mersenne seleccionado es mucho mayor y garantiza que ningún secreto quede fuera del rango.

3.2.2 Polinomio de grado 2

En nuestro caso se nos pidió realizar un secreto de Shamir (3,4) por lo cual un polinomio de grado k - 1 (k = 3) garantiza que se necesiten 3 o mas puntos para poder reconstruir el polinomio de Lagrange y poder acceder al secreto que se encuentra en $f(0)$. El caso de tener 4 llaves se genera una redundancia pero es útil en caso de que una llave se pierda.

3.2.3 Generación del Secreto

Para la generación de la clave secreta (secreto) utilizamos la librería secret de Python que genera aleatoriedad a través del sistema operativo siendo menos predecible que usar la librería random que a través de análisis complejos se puede obtener el valor. Ademas de que la librería encaja perfectamente con los bytes solicitados en el enunciado (32 bytes/ 256 bits) según la documentación.

3.2.4 Reconstrucción del Secreto

La reconstrucción se realiza mediante interpolación de Lagrange sobre el campo finito definido por el polinomio PRIME. Como el polinomio es de grado 2, se necesitan exactamente 3 puntos (Llaves). Evaluando el polinomio interpolado en $x = 0$ se obtiene el secreto original.

$$f(0) = \sum_{i=0}^{k-1} y_i \cdot L_i(0)$$

3.2.5 Simulacion

En el código encontraremos una función "simulacion(f1, f2, f3, f4, secreto)" esta se encarga de simular todos los casos posibles con 1 2 3 y 4 partes para poder acceder al secreto, este realiza interpolación de lagrange implementado en la función "interpolacion_lagrange(puntos)" para luego comparar si existe un match entre el secreto original y el obtenido con la interpolación de puntos

4 Interrogatorio

4.1 A) El Orden de los Factores

¿Por qué es vital firmar el mensaje antes de cifrarlo?

Porque asi la firma queda bajo la proteccion del cifrado, de esta forma no se puede modificar la identidad asociada a la firma ni tampoco conocer esta identidad sin saber la clave para descifrar. Si se hiciera de la forma inversa cifrando y luego firmando, alguien podria "desfirmar" el mensaje con la clave publica del emisor y luego firmar con su propia clave privada.

4.2 B) La Entropía de los Nombres

¿Cómo garantiza el SALT que no se puedan pre-calcular llaves?

...

4.3 C) El Error del Punto Flotante

¿Por qué la aritmetica decimal rompe el esquema de Shamir?

La interpolación de Lagrange calcula el secreto f(0) con una fórmula que involucra divisiones, multiplicaciones y sumas de números demasiado grandes o demasiado pequeños. Computacionalmente cada numero tiene un error de redondeo debido a los estándares (IEEE 754), cuando estos números se operan los errores de redondeo se se amplifican haciendo que el calculo de f(0) se impreciso y puede llevar a errores a la hora de volver a obtener el Secreto haciendo que el secreto original sea inservible.